4-2

Langfuse 기반 Observability & Evaluation

학습 목차

- **Langfuse**
  - Langfuse
  - Trace
  - `CallbackHandler`
  - `@observe`
- **Evaluation**
  - Rule-based Evaluation
  - LLM-as-a-Judge

© Scopelabs. All Rights Reserved

Langfuse

Langfuse (1)

Langfuse는 LLM 애플리케이션을 관찰하고 평가하기 위한 오픈소스 Observability 플랫폼이다.

- 프롬프트, 모델 호출, 도구 호출, 지연시간, 비용, 오류를 Trace 단위로 기록한다.
- LangChain, LangGraph, OpenAI SDK, Python SDK와 연동해 실행 흐름을 추적할 수 있다.
- 기록된 Trace를 바탕으로 디버깅, 비용 분석, 사용자 피드백, 품질 평가를 수행한다.
- 클라우드로 사용할 수도 있고, Docker Compose 기반으로 자체 서버에 설치할 수도 있다.

Langfuse (2)

LLM 애플리케이션은 같은 입력처럼 보여도 내부 실행 경로가 복잡하게 달라질 수 있다.

- 어떤 프롬프트가 실제로 모델에 전달되었는지 확인해야 한다.
- 어떤 도구가 어떤 입력으로 호출되었는지 확인해야 한다.
- 어느 단계에서 지연시간과 비용이 발생했는지 확인해야 한다.
- 실패한 응답이 모델 문제인지, 컨텍스트 문제인지, 도구 문제인지 분리해야 한다.

> **Trace는 "무엇이 잘못됐는가"를 감으로 추측하지 않고 실행 기록으로 확인하게 만든다.**

Trace

Trace는 LLM 애플리케이션의 실행 과정을 단계별로 기록한 데이터이다.

- 프롬프트 입력
- 모델 호출
- 도구 호출
- LangGraph 노드 실행
- 각 단계의 입력, 출력, 지연시간, 오류

Trace를 보면 하나의 사용자 요청이 내부에서 어떤 실행 흐름으로 처리되었는지 재구성할 수 있다.

`CallbackHandler`

`CallbackHandler`는 LangChain과 LangGraph 실행 이벤트를 Langfuse Trace로 보내는 연결 지점이다.

- Chain, Runnable, Tool, LLM 호출 이벤트를 자동으로 수집
- `config`에 `callbacks` 속성 중 하나로 전달
- `with_config`를 사용하면 기본 설정으로 적용

```
langfuse_handler = CallbackHandler()

response = chain.invoke(
    {"question": "Langfuse가 뭐야?"},
    config={"callbacks": [langfuse_handler]},
)
```

@observe

`@observe()`는 일반 Python 함수를 Langfuse Trace 또는 Span으로 기록하기 위한 데코레이터이다.

- LangChain 밖에서 실행되는 사용자 정의 함수를 추적할 수 있다.
- 검색, 후처리, 평가, 파일 생성처럼 프레임워크 바깥의 로직을 Trace에 포함할 수 있다.
- 하나의 요청 안에서 LLM 호출과 일반 Python 처리를 함께 관찰할 수 있다.

```python
from langfuse import observe

@observe()
def normalize_query(query: str) -> str:
    return query.strip().lower()
```

Evaluation

Evaluation

Evaluation은 응답이 좋은지 나쁜지를 점수로 남기는 과정이다.

- 트레이싱만으로는 답변 품질을 알 수 없다.
- Score를 남기면 품질 추세를 정량적으로 볼 수 있다.
- 규칙 기반 평가와 LLM 평가를 함께 사용할 수 있다.

Observability가 실행 기록을 남긴다면, Evaluation은 그 실행 결과의 품질을 수치와 근거로 남긴다.

Rule-based Evaluation

Rule-based Evaluation은 코드로 명확히 판단할 수 있는 조건을 평가하는 방식이다.

- 예시 포함 여부, 특정 키워드 포함 여부, 답변 길이 등

규칙 기반 평가는 단순하지만 안정적이다.

- 빠르고 저렴하다.
- 재현 가능성이 높다.
- 정성적 품질 판단에는 한계가 있다.

```
def has_sources(answer: str) -> float:
    return 1.0 if "출처" in answer or "http" in answer else 0.0

score = has_sources(answer)
```

LLM-as-a-Judge (1)

LLM-as-a-Judge는 별도 LLM이 응답 품질을 평가하는 방식이다.

- 답변의 유용성
- 질문과의 관련성
- 근거 충실성
- 설명의 완성도

규칙으로 판단하기 어려운 정성적 품질을 평가할 때 사용한다.

```
judge_prompt = ChatPromptTemplate.from_messages([
    ('system', '아래 답변의 품질을 0.0~1.0 사이 숫자 하나만 출력하세요. 다른 텍스트는 출력하지 마세요.'),
    ('human', '질문: {question}\n답변: {answer}'),
])
judge_chain = judge_prompt | llm | StrOutputParser()
```

LLM-as-a-Judge (2)

LLM Judge를 사용할 때는 평가 기준을 명확히 작성해야 한다.

- 평가 기준을 구체적으로 작성
- 점수 범위를 고정, 근거 없는 높은 점수를 방지
- 가능하면 규칙 기반 평가와 함께 사용